

动捕穿缝规划与控制执行

当前实现原理、外部数据接入、FSM 与代码结构

项目：many_controllerV3 记录日期：2026 年 10 月 4 日
对象：当前工作区；对比基线：Git HEAD 82c3c77

本文描述**现有代码实际做了什么**，并记录尚未解决的问题。它不是已经完成实飞验收的证明，也不是对论文全部实验的原样复现。初稿为文档整理；本修订已依据指定实飞日志同步更新动捕话题配置、父坐标系锁定及输入诊断，不改变优化问题或控制算法。

1 功能范围与阅读结论

系统在 CMD 模式保持悬停，用户选择本次穿过前几个窗框；规划器读取窗框刚体位姿与几何配置，构造一条从当前悬停位置出发、按顺序通过选定开口、到固定往返端点的连续轨迹。通过规划器检查后，发送节点把轨迹离散成带时间的 p, v, a 样本并一次性上传。控制器重建连续轨迹、独立检查，进入 READY 后等待用户输入 e 执行；结束后仍在 CMD 保持悬停，可以继续规划返程。

当前优化是上游 MINCO 稀疏参数化 + 穿框几何惩罚 + 动力学和静态电机分配惩罚，求解后再做密集采样检查。MINCO 是轨迹表示与梯度传播方法；外层约束、任务管理、ROS 接口和执行逻辑由本项目实现。当前没有房间长宽、墙壁或地面高度约束，没有额外路径长度或全局单调前进惩罚。

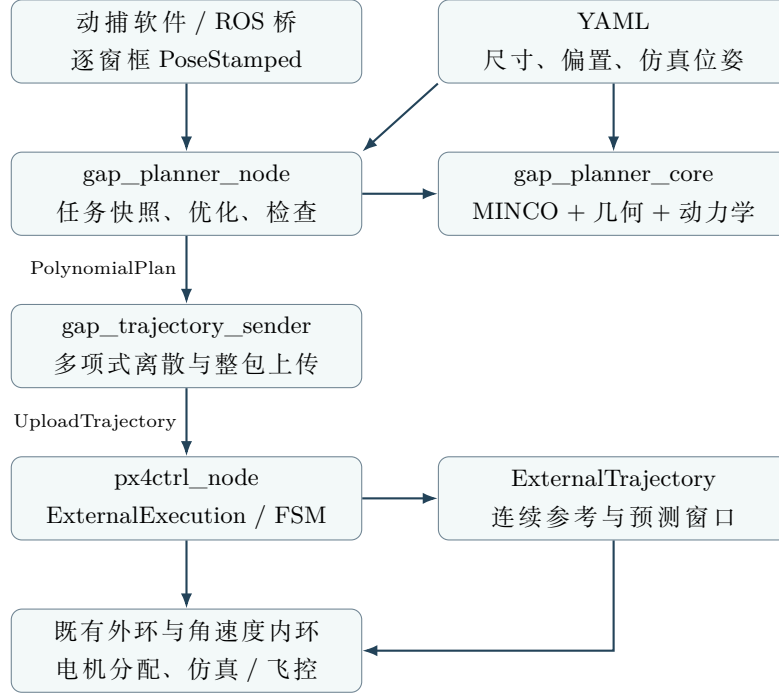
当前实机输入是每个窗框的 geometry_msgs/msg/PoseStamped。消息给出刚体的位置与姿态，**不包含标记球、窗框形状、开口尺寸和厚度**。这些几何量以及刚体系到开口系的偏置由 YAML 给出。动捕软件负责从标记球识别刚体，本项目负责接收、坐标变换和构造几何约束。

要回答的问题	当前答案
是否会直接优化姿态？	不设独立姿态决策变量；由位置轨迹的导数恢复姿态，机体外包络与窗框的约束反向影响轨迹和姿态。
何时接收和执行？	在 CMD 保持状态接收；上传成功仅表示进入异步检查，READY 后仍需每一航段显式启动。
为什么不是纯 MINCO？	MINCO 不自动定义障碍物、推力或电机约束，这些必须由应用层构造。
是否已达到论文几毫秒？	没有。现有日志单次外层求解为数十至数百毫秒，并可能多次恢复可行性。
最新失败是否已修复？	没有。密集检查发现动态峰值越界后，当前加密机制未把这些峰值时刻完整反馈给优化器，详见第12节。

目录

1	功能范围与阅读结论	1
2	总体数据流与模块边界	3
3	外部动捕数据如何接入	4
4	任务、端点与仿真场景	6
5	窗框与机体几何约束	8
6	MINCO 优化问题的实际形式	9
7	离散上传、连续重建与控制器数据结构	12
8	FSM：等待、执行和连续往返	13
9	ROS 接口与参数接入清单	14
10	本次功能引入后的项目结构与代码改动	16
11	构建、运行与接入核查	19
12	求解计时、已知失败与验证范围	20
13	当前体系的适用边界	22
A	文档复现与源码定位	23

2 总体数据流与模块边界



图中省略了反馈箭头：控制器以 `/gap/execution` 向规划器、控制台和仿真器报告状态；规划器以 `/gap/scene` 提供场景有效性心跳。实际无人机反馈仍由既有 PX4 消息链路进入控制器。

2.1 GCOPTER 的使用方式

`3rdpart/src/gcopter/` 是独立 Git 子模块，来源为官方仓库 [3]，当前固定提交为

`e0444f6d47b84f972ced91746b05feb36ce1fd4f`

本项目直接使用其中的 `minco::MINCO_S3NU`、`lbfgs.hpp` 和凸包相关头文件，不启动上游 ROS 演示节点，不修改上游 MINCO 的数学实现。`SparseOptimizer` 在这些基础上定义当前穿缝任务。

规划与控制在运行时分属不同 ROS 节点，但还存在明确的编译期依赖：`gap_planner_core` 链接控制器导出的 `px4ctrl::px4ctrl_reference`，复用参考解算与执行检查，避免两端使用不同飞行动力学约定。因此目前并非完全与控制器工程无关的通用规划库。以后若进一步解耦，可把共同参考库移动到独立包；这属于后续建议，尚未实施。

2.2 与论文和相关开源实现的关系

汪哲培博士论文第 5.6 节及 TRO 论文的 SE(3) 实验，采用整机椭球与凸安全走廊的包含约束，通过微分平坦性关联姿态与平移轨迹 [1, 2]。对半空间 $a^T x \leq b$ ，椭球包含约束可写成

$$a^T p + \|DR^T a\|_2 \leq b,$$

其中 D 为椭球半轴对角阵。Fast-Racing 使用机体几何顶点表达全机约束，其思想可写成对每个顶点 b_l 检查

$$a^T (p + Rb_l) \leq b.$$

当前默认凸包模型借鉴此表示 [4]，但当前有限窗框 SAT 构造、三段一航段的结点安排和 ROS 执行协议是本项目的实现，不是直接调用 Fast-Racing 完整规划流程。Fast-Perching 可作为 MINCO 应用层组织的相关参考 [5]，也不意味着本项目采用其相同边界条件或求解器配置。

论文中存在动力学约束，并非只优化光滑性与碰撞。当前与论文实验的差异包括机体模型、环境表达、采样密度、附加电机可执行性检查、气动近似、初值和实现开销；不能只因使用同一个 MINCO 头文件就继承论文的计时和成功率。

3 外部动捕数据如何接入

3.1 指定实飞记录中的实际消息

本次直接解析了 `flight_20261001_203447_613650_bc3a4984.ulg`，文件位于 `datalog/realflight/10012037TOP/`。记录器内嵌的 ROS 类型元数据和 2074 条实际样本确认：

- 原始动捕话题为 `/sun1/pose`，类型为 `geometry_msgs/msg/PoseStamped`，带位置、四元数及 ROS 时间戳。这是飞机刚体，日志中没有记录窗框刚体话题。
- 转给飞控的话题为 `/fmu/in/vehicle_visual_odometry`，类型为 `px4_msgs/msg/VehicleOdometry`。逐样本比较，位置严格满足 $(x, -y, -z)$ 变号映射，四元数满足 $(w, x, -y, -z)$ ，最大四元数分量向量误差约 5.98×10^{-8} 。
- 约 31.24 s 中记录 2074 条原始位姿，记录端观测平均频率约 66.35 Hz，时间戳全部递增。接收 ROS 时间减源时间戳的中位数约 7.73 ms，最大约 77.49 ms；它不是跨机器严格标定后的纯网络时延。
- 旧记录器明确省略字符串 `header.frame_id`，因此不能从此 ULog 得到实际父坐标系名称，更不能断言其值为 `mocap`。

当前消息类型与记录一致，**无需改成 `VehicleOdometry`**。窗框默认话题改为相同形式的 `/gap_1/pose` 等，实际名称仍在 YAML 中设置；这些名称是待与现场对应的窗框示例，不是从日志中发现的新刚体名。原始位姿经 YAML 世界标定后用于窗框，不重复执行飞机桥的 NED/FRD 变号。

`mocap.frame_id` 默认改为空字符串：第一条通过时间戳和位姿检查的消息确定父坐标系标签，随后所有窗框必须保持同一标签；第一条标签本身为空时也会锁定。已知现场标签时，可在 YAML 中显式填写并严格检查。自动锁定只识别标签，**不会求解外参或证明坐标原点一致**。不合法消息不会锁定标签，也不会更新上次有效开口位姿；拒收原因会限频记录并出现在缺失/过期诊断中。

日志核对结果和源文件 SHA-256 保存于 `docs/gap_mocap_log_reference.json`。源位姿到开口的实现集中在 `include/gap_planner/mocap_input.h`，由规划节点调用。新增测试使用日志首条数值位姿及非单位标定验证转换顺序、同父坐标系约束和异常数据拒收，测试中的父坐标系字符串是人工构造值。

本次构建及 4 项动捕输入、8 项几何测试通过。隔离 ROS 域的进程间测试曾验证原始位置不变号、错误标签拒收及新任务读取移动后的刚体；随后重跑出现 DDS 服务响应超时。因此这部分证据不作为稳定运行或实机接入验收，现场仍应按实际话题核对。

3.2 三个坐标系和一条变换链

统一使用列向量和主动旋转。定义 R_{AB} 把 B 系中的向量变换到 A 系， p_{AB} 为 B 原点在 A 系的位置：

- W：控制器局部世界系，穿缝接口命名为 `world_nwu`；
- M：动捕软件世界系，由 `mocap.frame_id` 标识；
- R：窗框刚体系，由动捕软件标定，原点可能在球架或窗框边缘；
- G：开口坐标系，原点为净开口中心，+X 为指定穿越方向、Y 为开口宽度、Z 为开口高度；
- B：无人机机体系，顶点与惯量使用控制器的前、左、上约定。

PoseStamped 提供 p_{MR}, q_{MR} ；YAML 提供全局标定 p_{WM}, q_{WM} 与每个窗框的 p_{RG}, q_{RG} 。最终开口位姿为

$$p_{WG} = p_{WM} + R_{WM}(p_{MR} + R_{MR}p_{RG}), \quad (1)$$

$$R_{WG} = R_{WM}R_{MR}R_{RG}, \quad (2)$$

$$q_{WG} = q_{WM} \otimes q_{MR} \otimes q_{RG}. \quad (3)$$

特别注意：opening_offset 在刚体系 $\mathbf{R}$ 表达，必须先随刚体旋转，不能直接作为世界系常数相加。opening_quaternion_wxyz 表示 G 到 R 的旋转，而不是其逆。代码的 Eigen 乘法顺序与上述公式一致。

3.3 四元数读取、归一化与姿态解释

ROS 消息按具名字段 orientation.x/y/z/w 存储；YAML 的四元数数组顺序则是 $[w, x, y, z]$ 。核心读取逻辑为

```
Eigen::Quaterniond q(msg.pose.orientation.w,
                      msg.pose.orientation.x,
                      msg.pose.orientation.y,
                      msg.pose.orientation.z);
q.normalize();
gate.center = world_translation + world_rotation *
              (rigid_body_position + q * opening_offset);
gate.rotation = (world_rotation * q * opening_rotation).normalized();
```

接收前先拒绝非有限数和范数小于 10^{-8} 的四元数。单位四元数 $q = (w, \mathbf{u})$ 对应

$$R(q) = (w^2 - \mathbf{u}^T \mathbf{u})I + 2\mathbf{u}\mathbf{u}^T + 2w[\mathbf{u}]_{\times}.$$

四元数 q 和 $-q$ 表示相同旋转，场景变化检测使用四元数的旋转角距离，而非四个分量直接相减。优化与几何变换使用旋转矩阵/四元数，不先转欧拉角再计算。

日志的 crossing_roll_deg 是无人机姿态的滚转角 $\text{atan2}(R_{32}, R_{33})$ ；crossing_tilt_deg 是机体推力轴相对世界竖直方向的倾角 $\arccos R_{33}$ 。它们不是窗框的滚转角，也不保证与窗框角度完全相等。窗框足够宽时，满足安全余量的较小倾角就是允许解。

3.4 刚体识别由谁完成

当前仓库没有从未标记的球点云建立刚体、匹配标记球或估计窗框尺寸的节点。应在动捕软件中建立各窗框刚体，确保其局部轴和原点稳定，由现有动捕 ROS 驱动或外部桥发布 PoseStamped。

若已有驱动输出 TransformStamped 或厂商自定义消息，需要外部适配为 PoseStamped：复制源世界系下的刚体平移、旋转与有效测量时间，设置正确的 frame_id。**本功能尚未新增特定厂商 SDK 桥接节点**；只改 YAML 中的话题名不能转换消息类型。

为解释动捕软件所做的“解算”，若已知同一刚体上的局部球点 b_j 与世界测量点 m_j 的对应关系，可求

$$\min_{R \in SO(3), t} \sum_j \|m_j - (Rb_j + t)\|^2.$$

令 $H = \sum_j (b_j - \bar{b})(m_j - \bar{m})^T = U\Sigma V^T$ ，则

$$R = V \text{diag}(1, 1, \det(VU^T))U^T, \quad t = \bar{m} - R\bar{b}.$$

这是已知对应点的刚体配准原理说明，**不是本仓库中的实现**。工程上应由动捕软件处理球点识别、遮挡和刚体有效性。PoseStamped 本身没有 tracking-valid 字段，上游丢失跟踪时不应持续发布“新时间戳的旧位姿”，否则当前新鲜度检查无法辨别。

3.5 世界系标定与旧代码命名的差异

控制器现有 `px4ctrl/src/input.cpp` 对 PX4 局部位置执行

$$p_W = (x_{PX4}, -y_{PX4}, -z_{PX4})^T, \quad v_W = (v_x, -v_y, -v_z)^T,$$

姿态四元数执行 $(w, x, y, z) \mapsto (w, x, -y, -z)$ 。其部分旧注释称 ENU，但此实现没有 ENU 所需的水平轴交换。本文按实际代码和新接口名称称为局部 NWU/前左上约定；局部 X 的实际航向仍由飞控估计器及实验标定决定。

因此，`mocap.world_*` 必须把动捕坐标映射到**控制器实际报告的位置和姿态坐标**，不能只按字符串“ENU”推断。若源真的是标准 ENU、目标真的是标准 NWU，同原点时旋转矩阵为

$$R_{WM} = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

而标准 NED 到标准 NWU 为 $\text{diag}(1, -1, -1)$ 。这两个矩阵仅说明轴约定，不代替现场原点、局部航向和飞控重置后的标定。

实际标定可选若干在 M 和 W 中均已知的对应点，拟合刚体变换并写入 YAML；再检查一个已知位置的开口中心、开口法向和宽高方向。现有节点不自动查询 TF，也不在线求全局标定。实机飞控原点重置时，固定标定需要重新核对。

3.6 无人机反馈和窗框动捕是两条链路

用途	输入 / 来源	当前责任边界
窗框几何	每个 <code>/gap_i/pose</code> ，名称可配置	规划器转换开口位姿；不估计无人机位置。
无人机位置、速度	<code>/fmu/out/vehicle_local_position</code> ，PX4 或仿真器	既有控制器输入转换、有效标志和状态机检查。
无人机姿态	<code>/fmu/out/vehicle_attitude</code> ，PX4 或仿真器	既有姿态输入链路。
飞机动捕进入飞控估计器	实验已有的外部定位/飞控融合链路	本次穿缝模块未实现厂商动捕到飞控的桥，不能由窗框话题替代。

3.7 时间、新鲜度和场景快照

实机接收使用 `SensorDataQoS`。消息 `header.frame_id` 必须与显式配置或首次有效消息锁定的标签完全一致；每个刚体时间戳必须递增，零、重复或倒序时间戳直接拒收。接收时先拒绝过期超过 0.3 s 或超前超过 0.05 s 的源时间戳；规划时再次要求源时间与接收时间新鲜。当前不做位姿外推、滤波或多刚体严格时间同步，使用每个选定刚体的最近有效测量。

选定数量后复制一份不可变场景快照；规划线程只读该快照，避免边优化边改变问题。后续新位姿若相对快照移动超过 0.01 m 或旋转超过 0.02 rad，场景被判为无效。规划结果发布前再次核对，控制器启动前也核对场景 ID 和心跳。该逻辑针对**静止窗框的一次规划**，不是运动窗框拦截或飞行中动态重规划。

4 任务、端点与仿真场景

4.1 数量选择和固定往返端点

`gate_order` 决定顺序，输入 N 表示选择前 N 个窗框，不是任意子集。设控制器保存的地面起飞原点为

p_{takeoff} ，则

$$p_{\text{home}} = p_{\text{takeoff}} + (0, 0, h_f)^T, \quad (4)$$

$$p_{\text{far}} = p_{\text{home}} + (d_x, 0, 0)^T. \quad (5)$$

其中 h_f 对应 `mission.flight_height`， d_x 对应 `mission.goal_offset_x`，当前分别为 1 m 和 4 m。偏移沿世界 **X** 轴，与飞行朝向或所选最后一个窗框无关。当前状态更接近 far 时判为返程，否则判为去程；起点始终取当前测得的悬停位置，终点取另一固定端点。

每次规划固定起终点速度、加速度为零。接收规划请求前要求 CMD 保持、反馈新鲜、起飞原点有效且当前速度不超过 0.12 m/s。这不是任意运动初态在线接续规划。

返程把所选窗框顺序逆转，并令每个开口旋转右乘 $R_z(\pi)$ ，从而反转穿越法向；对当前中心对称的矩形框，实体几何不变。控制台 `r` 实际上重新提交上次数量，去返程仍由当前位置判断，不是无条件翻转一个方向变量。

4.2 仿真位姿和“启动时没有窗框”

`simulation:=true` 时不订阅窗框动捕，而用 YAML 的 `sim_position` 与 `sim_rpy_deg` 作为刚体在 W 系的位姿：

$$R_{WR} = R_z(\psi)R_y(\theta)R_x(\phi), \quad p_{WG} = p_{WR} + R_{WR}p_{RG}, \quad R_{WG} = R_{WR}R_{RG}.$$

RPY 参数以度为单位；仿真位姿已经在控制器世界系中，不再应用 `mocap.world_*`。

MuJoCo 在加载时预分配所有窗框几何，以避免选择数量时重建模型或重置飞机。但 `GateSelection` 在启动时将全部窗框透明度和碰撞掩码置零，活动数量为 0；RViz 也没有选定任务的窗框显示。用户在 CMD 选择数量后，规划器先调用 `/gap/sim/configure` 激活前 N 个窗框，收到确认后再求解。场景配置在物理线程执行，执行中拒绝切换。这里的“预分配模型元素”和“激活可见、可碰撞窗框”是两件事。

实机选择数量不会使实际窗框消失。当前几何仅包含选定前 N 个框，未选框若仍可能挡路，属于未建模障碍；这需要实验布置与本次任务一致。

4.3 当前 YAML 快照及参数性质

以下是文档生成时读到的配置，不是先前全部测试使用的原始配置：

窗框	净宽/m	净高/m	厚度/m	框条宽/m	刚体位置/m; RPY/度
gap_1	0.50	0.22	0.04	0.05	(0, 0, 1.5); (60, 0, 0)
gap_2	0.90	0.34	0.04	0.05	(0.1, 0, 2.5); (-30, 0, 0)
gap_3	0.90	0.34	0.04	0.05	(1.1, 0, 1); (30, 0, 0)

三个开口偏置当前均为零，偏置旋转为单位四元数。当前仿真启动位置在 `launch` 中设为 $(-1.65, 0, 0.05)$ m，故默认 home 约为 $(-1.65, 0, 1.05)$ m，far 约为 $(2.35, 0, 1.05)$ m；实测起飞原点会引入小差异。

这些窗框并非一条等高直线：第一框在 $Z=1.5$ m，第二框在 $Z=2.5$ m。必须经过它们本身就要求爬升和下降。当前又没有房间边界限制，因此不能把路径绕行直接归因于“场地 4 m 太小”；仍需区分几何布置、倾斜所需加速度和非凸优化的局部解。

参数在节点构造时读取，现有实现没有完整的动态参数更新回调。修改 YAML 后应重启相关 `launch`；只执行 `ros2paramset` 不保证内部几何和优化选项同步更新。

5 窗框与机体几何约束

5.1 机体外包络

默认 `vehicle.body_model:polytope`，使用

$$\mathcal{B} = \text{conv}\{b_1, \dots, b_L\}, \quad \mathcal{B}_W(t) = p(t) + R(t)\mathcal{B}.$$

自定义顶点在机体系 B 表达，YAML 扁平数组依次存放 x, y, z ，需 4-64 个有限、非共面的点。没有自定义顶点时，使用八角点盒体：

$$b_l = (\pm(l + r_p), \pm(l + r_p), \pm H/2),$$

当前 $l = 0.1216$ m、 $r_p = 0.06475$ m、 $H = 0.10$ m，半尺寸为 $(0.18635, 0.18635, 0.05)$ m。该默认盒体较保守；真正自定义时应包含桨、机架、电池和动捕球，并与控制器机体原点一致。

椭球备选为 $\mathcal{B} = \{Du : \|u\| \leq 1\}$ ，默认半轴

$$D = \text{diag}(\sqrt{2}(l + r_p), \sqrt{2}(l + r_p), H/\sqrt{2}),$$

用于包住对应半径与高度的圆柱，并非从点集重新求解最小外接椭球。两类机体统一使用支持函数

$$h_{\mathcal{B}}(n) = \max_l n^T b_l \quad \text{或} \quad h_{\mathcal{B}}(n) = \|Dn\|.$$

5.2 有限厚度窗框：四个实体盒

设净开口宽高为 w, h ，厚度 d 、框条宽 b 。在 G 系中，左右竖条中心与半尺寸为

$$c_{\text{side}}^{\pm} = (0, \pm(w + b)/2, 0)^T, \quad s_{\text{side}} = (d/2, b/2, h/2 + b)^T;$$

上下横条为

$$c_{\text{cap}}^{\pm} = (0, 0, \pm(h + b)/2)^T, \quad s_{\text{cap}} = (d/2, w/2, b/2)^T.$$

再由 (p_{WG}, R_{WG}) 变换至 W 。规划器、MuJoCo 和 RViz 使用相同尺寸关系，`width/height` 均是**净开口**，不是外框尺寸。窗框是有限实体，不向整个空间延伸成无限墙，也不构建房间盒或长隧道。

5.3 穿越结点的开口包含约束

为每个框安排一个确切的 MINCO 结点 p_k ，参数化为

$$p_k = c_k + R_k e_y u_k + R_k e_z v_k.$$

于是 $e_x^T R_k^T (p_k - c_k) = 0$ 由参数化**严格成立**。对开口两轴 $a \in \{y, z\}$ 及符号 $\sigma \in \{-1, +1\}$ ，令 $n = \sigma R_k e_a$ ，要求

$$n^T (p_k - c_k) + h_{\mathcal{B}}(R(t_k)^T n) + m \leq \begin{cases} w_k/2, & a = y, \\ h_k/2, & a = z. \end{cases} \quad (6)$$

在优化时实际采用 $m + \delta$ ，默认物理余量 $m = 0.02$ m，数值预留 $\delta = 0.002$ m。另在结点惩罚 $n_x^T \dot{p}(t_k) < 0.05$ m/s；独立检查要求其严格大于零，确认指定方向穿越。

式(6)约束的是整个机体在开口平面上的投影，因而比“仅要求当时与平面相交的截面能通过”更保守。它有意使穿越姿态受到机体尺寸影响，但并未直接要求滚转角等于窗框滚转角。默认盒体、较大的余量与该投影条件都可能使问题更难。

5.4 全过程与框条的分离轴检查

任意时刻的机体与每根框条采用支持函数和分离轴定理 (SAT)。对单位候选轴 n ，机体在该轴上的最小投影与框条最大投影之差为

$$d_n = n^T(p - c) - h_B(-R^T n) - s^T |R_{\text{box}}^T n|, \quad d_{\text{SAT}} = \max_{n \in \mathcal{A}} d_n.$$

候选轴包含双方的面法向以及机体凸包边与框条边的叉积，正负方向均测试；同一平面三角剖分产生的对角线不作为真实几何边。优化要求 $d_{\text{SAT}} \geq m + \delta$ ，密集检查要求 $d_{\text{SAT}} \geq m$ 。

日志 clearance 是分离轴间隔证据，不是通用精确欧氏最近距离。对于完整的凸多面体 SAT 轴集，零余量时可判断是否相交；正余量条件是一种保守分离条件。椭球分支使用真实椭球支持函数但有限候选轴，因此可能不能为一些实际可分离姿态找到证据。先用包围球排除远处框条，以减少精确计算。

5.5 尺寸预检查能说明什么

`validateTask` 检查正尺寸、合法单位四元数、机体非退化和起终点。椭球使用最短半轴对应直径；凸包使用以顶点均值为中心、至最近凸包面的距离的两倍作为一个内接球直径下界。若最小净开口尺寸不大于该直径加 $2m$ ，提前拒绝。

对当前默认盒体，这个预检查阈值为 $0.10 + 2 \times 0.02 = 0.14$ m。它仅能排除明显不可能的尺寸，不能保证剩余任务动力学可行。旧日志 `Invalid/infeasiblelegateddimensions` 是此前实现的信息；当前代码已分为 `Invalidopening/rigid-bodypose`、内接直径说明和起终点框条相交等诊断。

6 MINCO 优化问题的实际形式

6.1 稀疏变量和多项式

选择 N 个框形成 $N + 1$ 个航段：起点到第一框、相邻框之间、最后框到终点。每个航段安排三段五次多项式，即

$$M = 3(N + 1), \quad p_i(t) = \sum_{r=0}^5 c_{ir} t^r, \quad t \in [0, T_i].$$

每个航段有两个自由三维结点，每个穿框结点有两个平面内自由度。再加 M 个时间变量，总维度为

$$n_x = 6(N + 1) + 2N + 3(N + 1) = 11N + 9.$$

因此一、二、三框分别为 20、31、42 个优化变量，对应 6、9、12 段。多项式系数不是外层独立变量，中间速度、加速度也不逐个作为外层决策量。

`MINCO_S3NU` 在给定结点位置、分段时间及首尾 PVA 条件后，解出满足连续性与最小三阶导数能量条件的系数。外层以这些稀疏变量改变整条曲线。首尾仅固定 p, v, a ，**没有显式要求首尾 jerk、snap 为零**；因此与静止悬停连接时不能声称所有高阶参考都连续为零。

时间正值通过无约束变量 $\tau_i \in \mathbb{R}$ 映射：

$$T_i = \begin{cases} 1 + \tau_i + \frac{1}{2}\tau_i^2, & \tau_i > 0, \\ (1 - \tau_i + \frac{1}{2}\tau_i^2)^{-1}, & \tau_i \leq 0. \end{cases}$$

程序另拒绝小于 10^{-4} s、超过 120 s 或非有限的分段时长，总时长在审核中也限制为 120 s。

6.2 初值

每个航段取起终点连线；邻近窗框的种子结点沿窗框法向布置，偏移距离为

$$d_0 = \text{clamp}(\|p_{\text{end}} - p_{\text{begin}}\|/3, 0.2, 0.6) \text{ m}.$$

航段初始时间由种子折线路长除以 $0.65v_{\text{max}}$ ，并下限为 0.8 s ，再均分三段。自由结点后续可离开这些种子位置；这些法向偏移不是固定隧道约束。当前没有多初值并行搜索，也没有全局最优保证。

6.3 目标函数和约束的实现性质

外层实际解的是带惩罚的无约束问题，可概括为

$$\begin{aligned} \min_x J(x) = & \frac{1}{g^2} \sum_{i=1}^M \int_0^{T_i} \|p_i^{(3)}(t)\|^2 dt + \rho_T \sum_i T_i \\ & + \lambda \left[\sum_{i,k} w_{ik} \Phi(z_{ik}) + \sum_{j=1}^N \Phi_{\text{gate}}(z_j) + \sum_{(i,\alpha) \in \mathcal{E}} \Phi(z_i(\alpha T_i)) \right]. \end{aligned} \quad (7)$$

其中 z 包含由同一多项式计算的 p, v, a, j, s 与恢复出的姿态、角速度等； ρ_T 当前为 100。每项违反量先按相应物理尺度归一化，再用平滑单侧函数

$$\phi_\mu(u) = \begin{cases} 0, & u \leq 0, \\ u^2/(2\mu), & 0 < u < \mu, \\ u - \mu/2, & u \geq \mu, \end{cases} \quad \mu = 0.01.$$

例如速度项的输入为 $\|v\|^2/(0.995v_{\text{max}})^2 - 1$ ，几何项输入为 $(m + \delta - d_{\text{SAT}})/0.1$ 。这里的硬等式由参数化消去，其他几何/动态限制是**软惩罚优化** + **独立拒绝式检查**，并非通用硬约束 NLP 求解器保证满足全部不等式。

每段基础采样为 40 个均匀区间、41 个结点，用梯形权重近似时间积分。精确穿框结点另加开口与方向惩罚；审核反馈的局部加密点另加惩罚，不乘同样的积分步长。因此(7)是对实际离散实现的表达，不能将加密项误称为严格连续积分。

限制	优化中如何加入	独立检查 / 说明
穿框平面	结点二维参数化，严格满足	精确结点检查平面误差 $\leq 10^{-6} \text{ m}$ 。
开口内机体投影	支持函数四个不等式，含 $m + \delta$	精确结点要求余量不小于 m 。
穿越方向	结点法向速度下限 0.05 m/s 的软惩罚	法向速度必须为正。
有限框条碰撞	全部选定框条的 SAT 间隔惩罚	约 1 ms 间隔检查全部轨迹。
平移速度	模长上限，优化用 $0.995v_{\text{max}}$	审核上限为 $1.001v_{\text{max}}$ 。
机体角速度	模长上限及控制器逐轴上限	模长容差系数 1.001；执行检查另外检查逐轴值。
倾角	$\cos \theta$ 形式，优化预留 0.002 rad	审核允许 10^{-4} rad 数值容差。
推力加速度	上下界，优化各留约 0.5% 余量	规划审核上下界系数分别 0.999、1.001；执行审核另用电机导出的范围。
角加速度	模长上限，优化用上限的 0.995	执行检查按控制器限制。
逐电机推力	静态分配后各电机上下限，预留 $0.02mg/4 \text{ N}$	执行检查逐电机名义推力。

限制	优化中如何加入	独立检查 / 说明
房间、地面、全局单 调 X	未加入	外部模式关闭最低高度检查；路径长度和后退距离只报告，不作为额外成本。

6.4 如何由平移轨迹恢复姿态与控制量

固定水平航向辅助向量为 $h = (\cos \psi, \sin \psi, 0)^T$ 。先由

$$z_0 = \frac{a + ge_3}{\|a + ge_3\|}, \quad y_0 = \frac{z_0 \times h}{\|z_0 \times h\|}, \quad x_0 = y_0 \times z_0$$

得到 $R_0 = [x_0, y_0, z_0]$ 。然后按现有控制器的一次气动补偿计算

$$v_b = R_0^T v, \quad (8)$$

$$a_{\text{aero},b} = -D_a v_b + k_h (v_{b,x}^2 + v_{b,y}^2) e_3, \quad (9)$$

$$a_f = a + ge_3 - R_0 a_{\text{aero},b}, \quad f = m \|a_f\|. \quad (10)$$

以 $a_f / \|a_f\|$ 为新机体 Z 轴、 R_0 的 X 轴为辅助方向，重新构造姿态 R 。这是与控制器对齐的一次修正模型，不是对含姿态气动力的完整隐式动力学方程作精确求解。无阻力时退化为常用平坦性映射。

由同一曲线的 jerk、snap 求旋转导数，进而

$$\omega = \text{vee}(\text{skew}(R^T \dot{R})), \quad \alpha = \text{vee}(\text{skew}(R^T \ddot{R})).$$

几何约束通过 $R(a, v)$ 影响位置轨迹：要产生倾角，就需要相应水平加速度；要在狭缝前倾斜、穿过后恢复，又受到角速度和电机能力限制。这是轨迹可能先偏移再加速回穿的物理来源之一。

6.5 电机可执行性与梯度

采用机体惯量 J 和刚体转动方程

$$\tau = J\alpha + \omega \times (J\omega), \quad \begin{bmatrix} f \\ \tau \end{bmatrix} = Bu,$$

其中

$$B = \begin{bmatrix} 1 & 1 & 1 & 1 \\ b & -b & -b & b \\ -a & -a & a & a \\ k & -k & k & -k \end{bmatrix}, \quad a = l \cos \beta, \quad b = l \sin \beta, \quad k = c_q / c_t.$$

求 $u = B^{-1}[f, \tau^T]^T$ 后施加各电机推力限制。这是名义静态分配检查，没有建模电机响应时延、反馈修正所需的全部余量或柔性桨效应。

`project_flatness.h` 实现与控制器一致的可求导映射：高阶时间量通过 jet 传播，局部输入梯度使用自动微分；没有在正常优化循环中对整条轨迹作有限差分。几何支持函数给出活动顶点与分离轴分支的梯度，再累加到多项式系数和时间偏导，调用上游 `minco_.propogateGrad(...)` 将梯度传播到稀疏结点和分段时间，最后乘二维开口映射与正时间映射的导数。

支持顶点与 SAT 活动轴切换会带来分段光滑性，局部梯度正确不等于整个非凸问题无局部极值。MINCO 的优势是消去大量系数并高效传播梯度，不能消除复杂几何评估和多次外层迭代的成本。

6.6 求解、加权和审核循环

1. 建立有限框条、检查起终点静止机体不碰框，生成稀疏初值；
2. 以初始惩罚权重 10^4 调用 L-BFGS：记忆长度 32、单次最多 500 次迭代、`past=5`、`delta=1e-5`、`g_epsilon=1e-6`；
3. 以不大于约 1 ms 的分段采样间隔进行几何与动态审核，精确检查穿框结点；之后复用控制器参考库检查角加速度和电机分配；
4. 审核通过立即返回。否则记录失败原因、添加部分局部采样点，将权重乘 10（上限 10^6 ），保留当前稀疏变量作热启动；
5. 最多 8 次调用，受 `solve_budget` 约束。默认 5 s，第一次调用使用剩余预算的三分之一；迭代回调检查期限，审核和一次计算本身可能带来额外耗时。

求解器返回码只表示数值算法停止原因。特别是上游 `lbfgs_status=1` 为 L_BFGS_STOP，对应目标变化判据停止，**不是所有约束已满足**。可发布的轨迹以独立审核结果为准。

7 离散上传、连续重建与控制器数据结构

7.1 多项式消息与发送策略

规划器通过 `/gap/polynomial` 发布 `PolynomialPlan`，含世界系、轨迹 ID、场景 ID、参考模型版本、每段时长和每段 3×6 系数。系数按 x、y、z 三行展开，每行按局部时间升幂存储： $c_0, \dots, c_5$ 。该话题为 `reliable`、`transient-local`、深度 1。

发送节点默认 `sample_dt=0.005 s`，每段取 $n_i = \lceil T_i / \Delta t \rceil$ ，在 kT_i/n_i 求 p, v, a ；段间共享端点只保留一次。因此全部原始多项式边界必定是样本点，实际采样间隔不超过配置值。

发送方式是通过 `/gap/upload` **一次上传整条带相对时间的轨迹**，不是按 200 Hz 依赖网络到达时刻逐点驱动飞机。DDS 可在传输层分片，飞行播放时钟仍由控制器激活时刻决定。消息最多 50000 点、总时长最多 120 s；接收端要求相邻样本间隔为 10 微秒至 100 毫秒，首点时间为零、时间递增、所有值有限、首尾速度和加速度近零。

7.2 五次 Hermite 重建

对相邻样本 (p_0, v_0, a_0) 与 (p_1, v_1, a_1) ，令时间间隔 h 、归一化局部时间 $u \in [0, 1]$ ，重建 $p(u) = \sum_{r=0}^5 c_r u^r$ 。定义

$$\begin{aligned} c_0 &= p_0, & c_1 &= hv_0, & c_2 &= \frac{1}{2}h^2a_0, \\ \Delta p &= p_1 - c_0 - c_1 - c_2, \\ \Delta v &= hv_1 - c_1 - 2c_2, & \Delta a &= h^2a_1 - 2c_2, \end{aligned}$$

则

$$c_3 = 10\Delta p - 4\Delta v + \frac{1}{2}\Delta a, \quad c_4 = -15\Delta p + 7\Delta v - \Delta a, \quad c_5 = 6\Delta p - 3\Delta v + \frac{1}{2}\Delta a.$$

每个样本区间都位于原五次多项式的一段内部，因此除浮点误差外能恢复该段原曲线，而非线性插值近似。遗漏原始分段边界会破坏这个性质。实际时间导数需乘 h^{-r} ；jerk、snap 由重建系数求得，不对离散姿态做差分。

7.3 数据如何进入控制器

`ExternalTrajectory` 实现既有 `Trajectory` 接口，保存时间数组、局部五次系数及参考飞行模型。`evaluate(t)` 先求 p, v, a, j, s ，再调用 `externalReference` 得到统一 `ReferencePoint`，其中含姿态、推力加速度、机体角速度

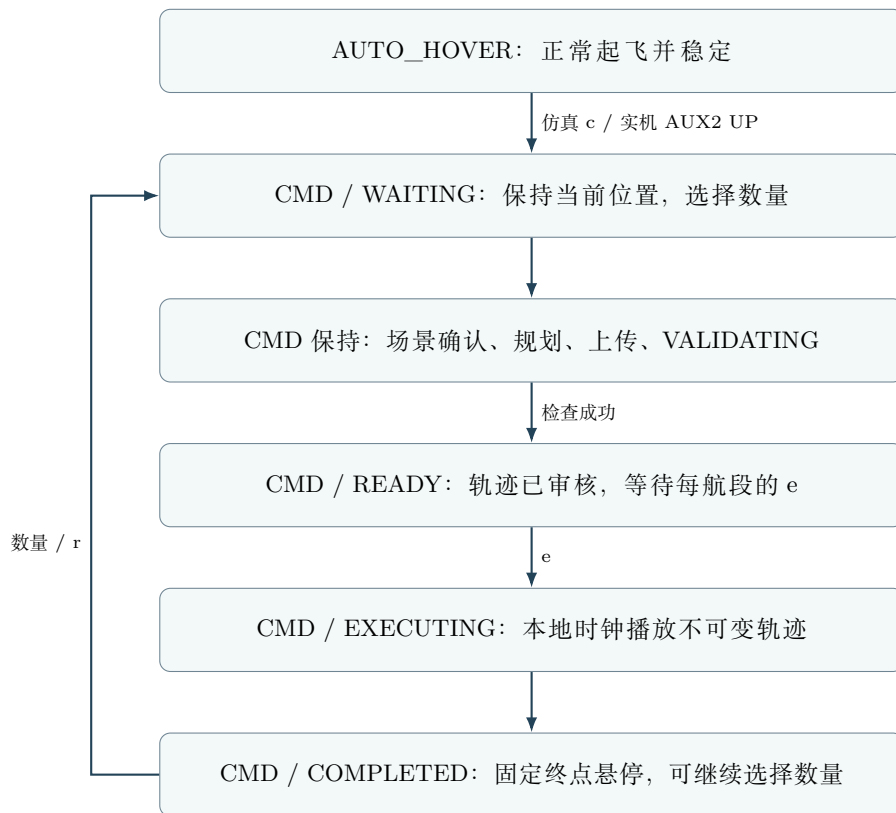
和角加速度。

`externalReference` 设置气动补偿已包含等标记，供后续控制路径识别；规划和执行复用同一模型，避免参考在两端被重复补偿。`FlightModel` 版本当前为 1，含质量、重力、线性阻力加速度系数、水平升力系数及航向。接收端检查质量、重力与气动系数匹配到 10^{-6} 量级，航向由轨迹给出。惯量和电机限制不在该消息内，而由控制器配置导入规划端并在控制端独立检查。

外部轨迹已经在绝对 W 系中，激活播放器时使用零平移、零附加偏航，不能再次按当前飞机位置平移。`TrajectoryPlayer` 在每个控制周期按真实预测时刻生成 `ReferenceWindow`，外环继续沿用原 OMMPC 或 `acados` 接口，下层角速度控制及电机分配流程保持原结构。PID 分支当前拒绝此外部 SE(3) 模式。

当 $t < 0$ 或 $t \geq T$ 时，`ExternalTrajectory` 返回端点静止悬停参考。由于优化仅固定首尾 PVA，高阶导数在接入/退出悬停处可能跳变，现有实现不应宣称具有完整 C^4 悬停接续。

8 FSM：等待、执行和连续往返



规划失败或控制器 REJECTED 时，飞机仍使用 CMD 悬停参考，可重新选择任务。CMD 等待是持续运行控制循环，不是阻塞等待 ROS 消息或求解线程。规划器和控制器的耗时审核都在独立工作线程运行。

8.1 进入 CMD 与状态检查

外部模式进入 CMD 时清理旧播放器，记录当前位置作为悬停目标，**不自动激活轨迹**。仿真 `/gap/enter_cmd` 只在 AUTO_HOVER、反馈有效且速度低于阈值时接受；实机不提供这个代替遥控器的入口，仍由飞手切 AUX2 UP。

上传服务仅在 CMD 保持、反馈有效、没有正在执行轨迹且审核器空闲时接受。`accepted=true` 意味着任务已排入异步验证，不等于 READY。验证以约 2 ms 间隔检查重建后的参考及静态电机分配；验证结果用代次编号隔离，离开 CMD 或结束任务后旧线程的结果不能重新激活旧轨迹。

8.2 READY 和启动条件

启动前要求场景 ID 与轨迹绑定一致、场景有效且心跳默认不超过 0.5 s，并要求当前位置距轨迹起点不超过 0.10 m、当前速度不超过 0.15 m/s、当前姿态与起点参考角距离不超过 0.15 rad。READY 过程中每次轮询仍核对这些条件，条件失效后丢弃待执行轨迹，要求重新规划。

用户输入 e 后只是设置启动请求，FSM 再次检查并激活本地轨迹，记录实际激活时刻。多项式发布时间、上传时间和执行起始时间是不同概念，不能拿消息传输时刻当作飞行相位。

8.3 完成、继续和结束

到达轨迹总时长后，FSM 清理执行对象，将轨迹终点设为 CMD 悬停目标，状态报告 COMPLETED。悬停判据中的 hovering 主要表示处于保持分支，不是测得速度严格为零；下一次规划还会检查实际速度。输入数量或 r 发起下一段，仍需等待 READY 并输入 e。

输入 f 在 CMD 保持时结束外部任务并回 AUTO_HOVER，不自动降落；执行中拒绝该服务。q 只退出控制台，不停止轨迹。原遥控器手动切换、降落和失效处理仍由既有 FSM 路径负责。

8.4 执行期间场景变化的现状

执行前场景变化会阻止启动。执行期间若动捕过期、窗框移动或心跳丢失，当前控制器只报告 executingimmutable-static-sceneplan，并继续已经激活的轨迹；没有新实现急停、避障逃逸或在线重规划。这是当前功能的明确边界，不能将执行前快照检查理解为飞行全过程的动态避障保证。

9 ROS 接口与参数接入清单

本节所有 gap_msgs 类型来自 src/utils/gap_msgs/，服务应按消息包生成后的真实类型调用。

话题 / 服务	类型	方向和作用
每框 YAML topic	geometry_msgs/msg/PoseStamped	外部动捕桥 → 规划器；仅实机订阅。
/gap/plan	gap_msgs/srv/PlanGaps	控制台 → 规划器，count 选择前 N 个。
/gap/polynomial	gap_msgs/msg/PolynomialPlan	规划器 → 发送器；已通过规划器审核的多项式。
/gap/upload	gap_msgs/srv/UploadTrajectory	发送器 → 控制器；整包 PVA 样本。
/gap/scene	gap_msgs/msg/SceneStatus	规划器 → 控制器；scene_id、valid、reason，约 10 Hz。
/gap/execution	gap_msgs/msg/ExecutionStatus	控制器 → 其他节点；FSM 状态、反馈、起飞原点、轨迹进度，约 10 Hz。
/gap/start	std_srvs/srv/Trigger	每航段显式启动，控制台 e。
/gap/finish	std_srvs/srv/Trigger	CMD 保持时结束外部任务，控制台 f。
/gap/enter_cmd	std_srvs/srv/Trigger	仅仿真，控制台 c。
/gap/sim/configure	gap_msgs/srv/ConfigureGates	规划器 → 仿真器；激活数量。

话题 / 服务	类型	方向和作用
/gap/sim/selected_count	std_msgs/msg/UInt32	仿真器发布实际活动数量，约 5 Hz。
/gap/planner_status	std_msgs/msg/String	人可读状态、失败原因及时间；可靠持久深度 1 发布。
/gap/markers	visualization_msgs/msg/MarkerArray	RViz 窗框、轨迹和机体包络；可靠持久深度 1。

9.1 核心参数表

参数完整命名在 `gap_planner.ros__parameters` 下；发送器参数在 `gap_trajectory_sender.ros__parameters` 下。

参数	当前值 / 单位	含义
<code>mission.goal_offset_x</code>	4.0 m	far 相对 home 的世界 X 偏移。
<code>mission.flight_height</code>	1.0 m	相对地面起飞原点的悬停高度。
<code>vehicle.height</code>	0.10 m	包含电池、动捕球的全高度。
<code>vehicle.body_model</code>	polytope	默认盒体或用户顶点凸包；另支持 ellipsoid。
<code>vehicle.vertices</code>	可选扁平数组	机体局部顶点，单位 m。
<code>planning.heading</code>	0 rad	固定几何航向辅助方向。
<code>planning.margin</code>	0.020 m	几何审核余量。
<code>planning.optimization_buffer</code>	0.002 m	优化额外数值预留。
<code>planning.speed_max</code>	2.0 m/s	平移速度模长上限。
<code>planning.rate_max</code>	6.0 rad/s	机体角速度模长上限。
<code>planning.thrust_min/max</code>	3.0 / 18.0 m/s ²	推力除以质量，不是 N 或油门。
<code>planning.tilt_max</code>	1.52 rad	相对竖直推力轴最大倾角，约 87.1 度。
<code>planning.time_weight</code>	100.0	时间项相对归一化 jerk 能量权重。
<code>planning.solve_budget</code>	5.0 s	全部恢复尝试共用的软预算。
<code>mocap.frame_id</code>	空字符串	自动锁定首条有效消息的父坐标系；非空配置则显式精确匹配。
<code>mocap.world_translation</code>	(0, 0, 0) m	P_{WM} 。
<code>mocap.world_quaternion_wxyz</code>	(1, 0, 0, 0)	q_{WM} ，单位四元数。
<code>mocap.timeout</code>	0.3 s	刚体时间戳与接收时间新鲜度。
<code>mocap.position_tolerance</code>	0.01 m	快照开口中心移动阈值。
<code>mocap.angle_tolerance</code>	0.02 rad	快照开口姿态变化阈值。
<code>gates.<id>.opening_offset</code>	三维位置 / m	p_{RG} ，刚体系下开口中心。
<code>gates.<id>.opening_quaternion_wxyz</code>	单位四元数	q_{RG} 。
<code>gates.<id>.sim_position/sim_rpy_deg</code>	m / 度	仿真刚体位姿。
<code>sample_dt</code>	0.005 s	发送器采样上界，不是网络播放频率。

质量、力臂、桨半径、重力、惯量、气动系数、机体逐轴角速度、角加速度和电机上下限由 launch 读取控制器 YAML 并传给规划器。控制器和规划器气动补偿开关必须匹配。电机推力上限由原电机映射和 trajectory.limits.motor_fraction 导出，launch 中的公式与当前控制器参数约定一致。

旧参数 planning.tunnel_length、planning.path_length_weight、planning.backward_weight 和 planning.penalty 已不参与当前稀疏规划；若传入会告警并忽略。不要用修改这些字段的方式期待改变现有解。

9.2 实机接入配置示意

以下仅展示应填写的字段层次；单位旋转与零偏置只有实际标定恰好如此时才正确。窗框完整尺寸仍需结合现有 YAML 填写。

```
gap_planner:
  ros__parameters:
    simulation: false
    mocap:
      frame_id: ""
      world_translation: [0.0, 0.0, 0.0]
      world_quaternion_wxyz: [1.0, 0.0, 0.0, 0.0]
      timeout: 0.3
    gate_order: [gap_1]
    gates:
      gap_1:
        topic: /gap_1/pose
        width: 0.50
        height: 0.25
        thickness: 0.04
        frame_width: 0.05
        opening_offset: [0.0, 0.0, 0.0]
        opening_quaternion_wxyz: [1.0, 0.0, 0.0, 0.0]
```

动捕适配节点应提供原始有效测量的 header.stamp，使其与 ROS 时间处于可比较时基；实机使用墙上时钟，仿真节点统一使用 /clock。仅修改坐标系名称不会执行坐标变换，也不应先在外部分转换到 W 后仍在规划器重复应用同一非单位标定。

10 本次功能引入后的项目结构与代码改动

10.1 相对原工程的结构变化

比较基线为 82c3c77 的原工程。功能代码目前存在新增/修改的工作区文件，不能把它们描述成已经合并到基线提交中。

```
many_controllerV3/
  3rdpart/src/gcopter/          # new upstream submodule
  src/utlils/gap_msgs/          # new ROS interfaces
  src/realflight_modules/
    gap_planner/                # new planning package
      include/gap_planner/
      src/
      scripts/gap_console.py
      config/gaps.yaml
      config/gaps_tilt_demo.yaml
      config/gaps.rviz
      launch/gap_flight.launch.py
      test/
```

```
px4ctrl/                                # existing control package
include/px4ctrl/external_trajectory.h    # new
include/px4ctrl/external_execution.h     # new
src/external_trajectory.cpp              # new
src/external_execution.cpp               # new
src/px4ctrlfsm.cpp                      # modified
src/trajectory.cpp                      # modified
src/uav_simulator/quadsim_mujoco/
quadsim_mujoco/gap_scene.py             # new finite frames / selection
quadsim_mujoco/quadsim_node.py          # modified integration
0913/gap_planner_mocap_minco_implementation_20261004.tex
```

方面	加入功能前	当前实现
轨迹来源	控制器既有解析轨迹、omtraj / CSV 等	保留原来源，新增外部穿缝轨迹类型 external。
任务输入	既有轨迹参数与 CMD 激活	外部控制台选择数量，读取本次刚体快照。
优化位置	原轨迹生成路径	独立规划节点调用 GCOPTER MINCO 基础库。
CMD 行为	进入后加载既有轨迹源	external 分支先悬停，READY 后显式启动，完成后可继续等待。
轨迹载体	Trajectory、ReferencePoint、ReferenceWindow 已存在	新 ExternalTrajectory 接入既有框架，并非重写整个控制器数据结构。
环境	原 MuJoCo 场景	新增参数化有限窗框和按数量激活。
模型一致性	既有控制器内部参考解算	导出公共参考库，规划器复用，同时上传模型版本与参数。

10.2 新增规划包的文件职责

下表路径相对 src/realflight_modules/gap_planner/。

文件	实现职责
include/gap_planner/core.h	Gate、Options、Piece、Result、PlanTiming、PlanError 和规划入口。
include/gap_planner/body_geometry.h	机体支持函数、旋转顶点梯度和凸包辅助接口。
src/geometry.cpp	默认箱体、顶点验证、可视化边及多项式导数求值。
include/gap_planner/frame_geometry.h	任务几何预检查、四根有限框条、SAT 分离和解析几何梯度。
src/frame_geometry.cpp	
include/gap_planner/project_flatness.h	与控制器一致的气动参考映射、高阶时间量及自动微分。
include/gap_planner/sparse_optimizer.h	稀疏结点/时间变量、初值、MINCO 能量、各项惩罚、梯度传播、L-BFGS、局部加密。
src/sparse_optimizer.cpp	
src/optimizer.cpp	规划入口、密集审核、恢复可行性循环及全部计时。
src/planner_node.cpp	刚体订阅与坐标变换、往返任务、不可变快照、异步求解、场景心跳及 RViz。
include/gap_planner/ros_conversion.h	Eigen、内部参考模型与 ROS 消息之间的转换。

文件	实现职责
include/gap_planner/mocap_input.h	依据实飞 PoseStamped 接口读取与转换刚体位姿，校验源时间、锁定公共父坐标系，并测试记录数值及非单位偏置。 接收多项式、保留边界离散、整包上传和上传确认耗时。 离线合成场景/YAML 规划检查及耗时输出。 c、数量、e、r、f、q 人机命令与状态显示。 模型参数联通及控制器、规划器、发送器、仿真、RViz 统一启动。
test/mocap_input_test.cpp	
src/sender_node.cpp	
src/plan_check.cpp	
scripts/gap_console.py	
launch/gap_flight.launch.py	主任务配置、演示配置和 RViz 视图。
config/gaps.yaml	
config/gaps_tilt_demo.yaml	
config/gaps.rviz	几何、参考映射和目标梯度等单元检查。 多场景往返离线回归及 ROS 闭环流程验证。
test/geometry_test.cpp	
test/planning_regression.py	
test/simulation_flow.py	
CMakeLists.txt	新包依赖、GCOPTER 头文件、可执行程序与脚本安装、测试注册。
package.xml	

10.3 控制器及仿真改动

下表路径以相应包目录为根；这些是功能引入后的当前差异，不表示本文又执行了一次代码修改。

文件	改动内容
px4ctrl/include/px4ctrl/external_trajectory.h	新增飞行模型、PVA 样本轨迹、五次重建及统一外部参考解算。
px4ctrl/src/external_trajectory.cpp	
px4ctrl/include/px4ctrl/external_execution.h	新增上传/启动/结束服务、异步检查、场景匹配、READY/EXECUTING 等执行状态。
px4ctrl/src/external_execution.cpp	
px4ctrl/include/px4ctrl/px4ctrlfsm.h	增加 external 管理对象；CMD 等待、逐段激活、完成悬停和回 AUTO_HOVER；仿真不再自动进入外部轨迹执行。
px4ctrl/src/px4ctrlfsm.cpp	
px4ctrl/include/px4ctrl/input.h	增加 PX4CTRL_SIMULATION 编译开关控制 SIMULATION、USE_WITHOUT_RC,原传感器坐标转换未因本功能重写。
px4ctrl/include/px4ctrl/trajectory.h	新增 enforce_minimum_altitude；external 关闭此项，其他模式默认保留；复用已有名义执行检查。
px4ctrl/src/trajectory.cpp	
px4ctrl/CMakeLists.txt	引入 gap_msgs；将 ExternalTrajectory 加入公共参考库并导出；编译 ExternalExecution，新增外部轨迹测试。
px4ctrl/package.xml	
px4ctrl/test/external_trajectory_test.cpp	外部参考重建、校验及模型一致性相关测试。
quadsim_mujoco/quadsim_mujoco/gap_scene.py	新增四盒窗框 XML 和 GateSelection 激活/隐藏逻辑。
quadsim_mujoco/quadsim_mujoco/quadsim_node.py	接入窗框场景文件、配置服务和状态订阅，发布活动数量，记录窗框接触。

文件	改动内容
quadsim_mujoco/package.xml	增加消息和相关运行依赖。
src/utils/gap_msgs/	新增 5 个 msg 和 3 个 srv: FlightModel、FlatSample、PolynomialPlan、SceneStatus、ExecutionStatus, 以及 PlanGaps、ConfigureGates、UploadTrajectory。
.gitmodules	注册上游 GCOPTER 子模块。
3rdpart/src/gcopter/	
PROJECT_GUIDE.md	补充外部模式、构建运行、几何与检查说明。
px4ctrl/TRAJECTORIES.md	

10.4 与此前中间版本的区别

本功能迭代过程中曾采用更重的通道/走廊构造和额外路径类成本；现有代码已经改为**稀疏穿框结点 + 有限实体框条**，废弃的 tunnel、path-length、backward 参数不再生效。这个中间过程与“原工程 Git 基线”不是同一比较对象：前表给出可从当前工作区和 HEAD 核对的工程差异，本小节只用于解释此前讨论中旧参数为何不再起作用。

11 构建、运行与接入核查

以下命令从仓库根目录执行，使用本工程已有 ROS 2 和第三方依赖环境；完整依赖安装仍按原工程指南。文档生成过程未重新执行飞行或规划性能测试。

11.1 构建并启动仿真

```
source /opt/ros/humble/setup.bash
git submodule update --init 3rdpart/src/gcopter
colcon build --packages-up-to gap_planner quadsim_mujoco \
  --symlink-install \
  --cmake-args -DCMAKE_BUILD_TYPE=Release -DPX4CTRL_SIMULATION=ON
source install/setup.bash
ros2 launch gap_planner gap_flight.launch.py simulation:=true
```

另一个已 source 工作区的终端执行

```
ros2 run gap_planner gap_console.py
```

待 AUTO_HOVER 稳定后，按 c、1（或其他数量）、等待 READY、e 的顺序。完成后 r 或重新输入数量，再等待 READY 并 e；结束用 f。

此前的 Noexecutablefound 应从脚本安装和当前终端所 source 的工作区排查。现有 CMake 已有

```
install(PROGRAMS scripts/gap_console.py DESTINATION lib/${PROJECT_NAME})
```

因此构建/安装新包并 source 后可检查

```
ros2 pkg executables gap_planner
ros2 pkg prefix gap_planner
```

若选择控制器类型，launch 的 controller_kind:=mpc 或 nmpc 必须与现有控制器编译选择一致；它负责导入对应模型参数，不在运行时替换已经编译的控制器。

11.2 实机模式

实机需用 `-DPX4CTRL_SIMULATION=OFF` 构建控制器及依赖它的规划包，运行时使用

```
ros2 launch gap_planner gap_flight.launch.py \
  simulation:=false \
  gap_params:=/absolute/path/to/gaps.yaml \
  controller_params:=/absolute/path/to/controller.yaml
```

编译开关和运行参数不一致会被拒绝。此 launch 不自动提供厂商动捕驱动、飞控通信代理或飞机外部定位融合节点，需使用实验现有输入链路。进入 CMD 使用实机遥控器；后续数量选择、READY 与 e 启动流程一致。

11.3 可独立检查的接口

```
ros2 topic info /gap_1/pose -v
ros2 topic echo /gap_1/pose --once
ros2 topic hz /gap_1/pose
ros2 topic echo /gap/execution
ros2 topic echo /gap/planner_status
ros2 service call /gap/plan gap_msgs/srv/PlanGaps "{count: 1}"
```

检查顺序应是消息类型和 frame_id、四元数与米制单位、时间戳新鲜度、W 系位置与法向、最后才是求解结果。上述规划服务要求已处于 CMD 保持；直接调用并不会跳过 FSM。

离线规划命令可绕过 ROS，定位几何/优化本身的问题：

```
ros2 run gap_planner gap_plan_check --yaml \
  src/realflight_modules/gap_planner/config/gaps.yaml \
  1 outbound \
  src/realflight_modules/px4ctrl/config/params.yaml
```

把 outbound 改为 return 可测返程。该离线程序从 YAML 仿真位姿构造场景，默认起点按仿真设置，**不读取实机动捕**；当前控制器 YAML 解析路径采用 mpc 参数分支，不能未经核对就当所有 nmpc 配置的等价基准。

12 求解计时、已知失败与验证范围

12.1 每项耗时代表什么

所有规划计时使用单调墙上时间 `steady_clock`，不受仿真时钟快慢影响。

日志字段	含义
setup_ms	输入和几何准备、初值与求解器构造。
stages_ms[k]	第 k 次完整 L-BFGS 调用时间；不是一次迭代。
optimize_ms	各次 L-BFGS 调用时间之和。
audit_ms	所有候选轨迹独立审核累计时间。
total_ms	规划 API 从调用至成功/抛出失败的总墙上时间。
evaluations/iterations	跨调用累计的目标评估次数/迭代回调次数；线搜索使评估次数通常更多。
retry[k]	第 k 个候选审核失败的原因，不表示它后来必然被修复。
upload_ack_ms	发送节点从调用上传到收到服务响应，响应可能只是已排队。
controller_validation_ms	控制器重建样本并独立审核的时间。

控制台从输入数量到 READY 的端到端等待还包含仿真配置服务、节点轮询周期、采样、传输和控制器审核；不能与规划 `total_ms` 或单次 `stages_ms` 混为一谈。`flight_duration` 是轨迹飞行时长，与计算耗时不同。

12.2 用户最新失败日志的定量解释

用户提供的失败记录为一框任务，6 段、20 变量：

```
setup_ms=0.13 optimize_ms=944.72 audit_ms=35.48 total_ms=980.41
stages_ms=[177.96,463.93,104.80,38.73,39.61,40.87,38.06,40.75]
lbfgs_status=[1,1,1,1,1,1,1,1]
clearance=0.021853 speed=2.15606 rate=6.00074
tilt=1.15566 thrust=[7.51438,15.1164]
```

按当前限制，最终 `clearance` 超过 0.02 m；角速度 $6.00074 < 6 \times 1.001 = 6.006 \text{ rad/s}$ ，在审核容差内；倾角与推力范围也未越过所列上限。明确未通过的是**速度**： $2.15606 > 2 \times 1.001 = 2.002 \text{ m/s}$ ，实际超过标称上限约 7.8%。审核并非仅因 6.00074 比 6 略大就拒绝。

第一轮只占优化累计时间约 18.8%；后续七轮合计 766.76 ms，占约 81.2%。但第二轮 463.93 ms 本身也在进行可行性恢复，不能将后七轮全部称为无效重复；最后六轮共约 302.83 ms (32.1%) 呈现非常接近的结果。第一轮 177.96 ms 也还不是论文所述几毫秒。

12.3 当前自适应加密缺口

当前 `src/optimizer.cpp` 在密集审核时统计了全程最大速度、角速度、倾角与推力范围，但没有保存这些极值所在时刻并传给 `refine`。它实际反馈的是：

1. 每段、每根框条接近或违反 $m + \delta$ 的最小间隔时刻；
2. 在前面的几何/动态审核已经通过后，控制器执行审核返回的首个失败时刻。

`SparseOptimizer::refine` 会在反馈时刻附近按当前时长添加约 $\pm 10 \text{ ms}$ 的 21 个采样点，并保存为分段归一化时间。如果速度峰值落在未覆盖的采样间隙，提高惩罚权重仍可能无法看到该处越界；权重到达 10^6 上限、没有有效新增点后，热启动很容易继续收敛到同一附近结果。

这是从现有代码和日志能够确定的结构性缺口；**该日志本身尚不能证明具体速度峰值一定发生在基础采样间隙**，还需记录峰值时间及该时间处的代价/梯度来完整定位。即使峰值落在采样点上，惩罚权衡和局部收敛也可能留下越界。本文记录问题，没有将它表述为已经修复。

后续有依据的改进方向是先将所有违反动态限值的极值时刻反馈到相同加密机制，输出约束分项成本、峰值时间、活动采样点及每轮改变量；在客观无进展时停止重复求解并说明原因。之后再针对热点减少几何/微分计算开销，比较首个可行解时间。以上是建议，不是当前已有功能。

12.4 与论文计算时间如何比较

博士论文表 5.1 的特定仿真任务报告 4.4–7.4 ms 的计算时间；对应的机体、通道、限值和走廊表达与当前场景不同 [1]。这说明 MINCO 方法在相应问题上能够达到几毫秒，而不是任意 MINCO 应用都具备这个性能。

当前每次目标评估包含各段采样、机体与四框条 SAT、高阶参考映射和电机分配，多次恢复可行性进一步增加成本。稀疏变量只有 20 个也不意味着每次函数评估很便宜。有效性指标应同时报告**第一次调用时间、首个通过审核解的总时间、成功率和相同物理阈值**，而不能用失败初解耗时替代可执行轨迹耗时。

12.5 已有验证证据及其边界

此前工作区已有的离线回归产物 `/tmp/gap_planning_regression_final.json` 记录 24/24 个有限场景通过，包括当时配置下的一至三框往返、悬停起点扰动、窗框小扰动和等高倾斜场景。部分历史计时如下，单位 ms：

当时配置的场景	优化累计	审核累计	总时间
1 框去程	425.14	15.51	440.73
2 框去程	1126.32	36.49	1162.90
3 框去程	1759.51	30.90	1790.50

这些历史配置中的第一框并非本修订时 YAML 的 (0,0,1.5) m、60 度、净高 0.22 m，不能据此宣称用户最新场景已经通过。临时日志路径也不是永久的版本化测试数据库。

此前 `/tmp/gap_sparse_actuator_sim3_retry.log` 记录三框去程、两框返程的完整 ROS 仿真，规划总时间约 1828.69 ms 与 750.02 ms。几何/梯度单元测试、外部参考测试和有限闭环场景为实现提供局部证据；它们不能覆盖任意布局，也不能推翻用户这次真实出现的求解失败。本次动捕接入修订已重新构建规划包并通过动捕输入与几何测试，没有新增优化性能测量或实飞测试。

13 当前体系的适用边界

这套实现已经把刚体位姿、有限窗框、机体包络、MINCO 轨迹、离散传输及控制器播放连成同一条可检查的数据链。仍需保留以下事实作为后续开发依据：

- 动捕接入要求已有刚体识别和飞机定位链路；没有自动窗框尺寸识别、自动世界系标定或厂商 SDK 桥。
- 优化非凸且依赖种子与离散采样；密集审核也是有限采样，不是连续时间形式化碰撞证明。
- 当前动态极值反馈缺口会造成重复失败，尚未达到论文的几毫秒性能。
- 模型中的整机投影包含和默认箱体可能保守；没有为每个任务人为指定必须滚转多少度。
- 不加入房间/地面约束是按本任务要求实现的选择；飞机仍必须按实际实验环境布置任务。
- 仅支持从近似静止悬停开始的逐段任务；高阶悬停接续、飞行中窗框移动处理、动态重规划尚未实现。
- 逐电机限制针对名义静态分配，不能当作包含感知误差和闭环跟踪误差的全系统保证。

参考文献

- [1] 汪哲培. 多旋翼无人机运动规划的几何方法. 浙江大学博士学位论文, 2022. 第 5.6 节、图 5.15–5.17 与表 5.1. 项目内文件: 0913/多旋翼无人机运动规划的几何方法-汪哲培.pdf.
- [2] Zhepei Wang, Xin Zhou, Chao Xu, Fei Gao. Geometrically Constrained Trajectory Optimization for Multi-copters. *IEEE Transactions on Robotics*, 38(5):3259–3278, 2022. DOI: [10.1109/TRO.2022.3160022](https://doi.org/10.1109/TRO.2022.3160022). 项目内同名 PDF.
- [3] ZJU FAST Lab. GCOPTER official source. <https://github.com/ZJU-FAST-Lab/GCOPTER>. 本项目固定提交见第 2 节，重点文件为 `gcopter/include/gcopter/minco.hpp` 与 `lbfgs.hpp`.
- [4] ZJU FAST Lab. Fast-Racing official source and associated paper. <https://github.com/ZJU-FAST-Lab/Fast-Racing>. 本文引用其整机几何顶点表达思路，不声明原样复现实验或运行其完整后端。

- [5] ZJU FAST Lab. Fast-Perching official source. <https://github.com/ZJU-FAST-Lab/Fast-Perching>. 相关实现: `src/traj_opt/src/traj_opt_perching.cc`.

A 文档复现与源码定位

本文件为独立 LaTeX，不依赖外部图片或其他章节文件。使用本机可用的 Fandol 中文字体和 XeLaTeX 编译：

```
cd 0913
xelatex -interaction=nonstopmode -halt-on-error \
    gap_planner_mocap_minco_implementation_20261004.tex
xelatex -interaction=nonstopmode -halt-on-error \
    gap_planner_mocap_minco_implementation_20261004.tex
```

源码快速定位可使用

```
rg -n 'world_translation_|opening_offset|PoseStamped' \
    src/realflight_modules/gap_planner/src/planner_node.cpp
rg -n 'SparseOptimizer::|hinge|propagateGrad' \
    src/realflight_modules/gap_planner/src/sparse_optimizer.cpp
rg -n 'refinement|max_speed|retry|stage_ms' \
    src/realflight_modules/gap_planner/src/optimizer.cpp
rg -n 'Upload|VALIDATING|READY|ready\(|activate|complete' \
    src/realflight_modules/px4ctrl/src/external_execution.cpp
```

页内代码路径均指项目源文件；对旧工程差异可结合 `gitdiffHEAD` 与新增文件清单核对。源码继续修改后，应同步更新本文的问题形式、接口契约与已知问题，不能仅沿用本报告日期作为新版本证据。